IMT Mines Alès
École Mines-Télécom

RÉPUBLIQUE FRANÇAISE
*Liberté*
*Égalité*
*Fraternité*

# Graphical Models and Bayesian Networks

## Advanced Machine Learning

**22,24/09/2026**

**Andon Tchechmedjiev** [1]

[1] SyCoIA, IMT Mines Alès

# Outline

1. Graphical models and joint distributions
2. Conditional independence
3. Inference: from a question to factor operations
4. Learning and checking a model
5. From probabilities to engineering decisions

# Graphical models and joint distributions

## A short formal language for uncertain systems

Let $X_1, \ldots, X_n$ be random variables, with possible values $x_i \in \mathcal{X}_i$.

### From variables to a probability model

A graph $G = (V, \mathcal{E})$ records the relationships used by the model.

The joint $P(X_1, \ldots, X_n)$ assigns a probability to each complete configuration.

For a Bayesian network, the graph specifies a product of local distributions:

$$P(X_1, \ldots, X_n) = \prod_i P(X_i \mid \mathrm{parents}(X_i)).$$

We will **represent** this product, **learn** its values and **infer** from observations.

## The same formalism serves different kinds of questions

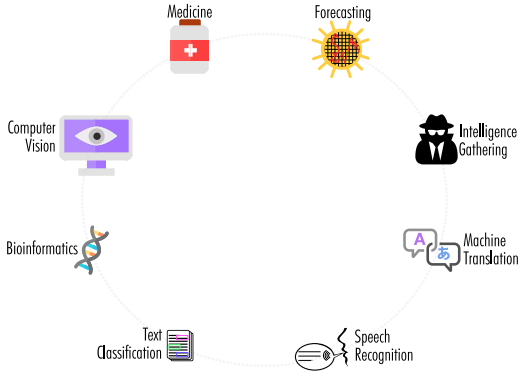
### A shared reasoning pattern

Several variables interact; only some are observed.

Use the observations to ask about a label, a future event or a hidden state.

Represent the joint, obtain its parameters, then compute a conditional probability.

## The same formalism serves different kinds of questions



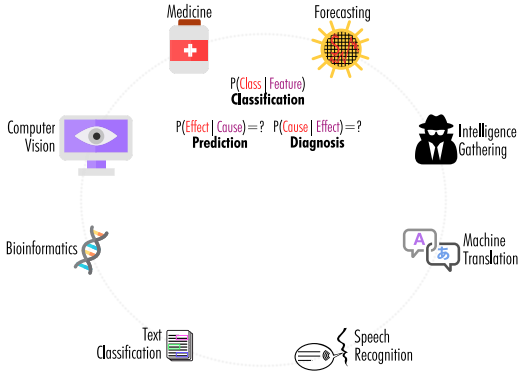

### A shared reasoning pattern

Several variables interact; only some are observed.

Use the observations to ask about a label, a future event or a hidden state.

Represent the joint, obtain its parameters, then compute a conditional probability.

## The same formalism serves different kinds of questions



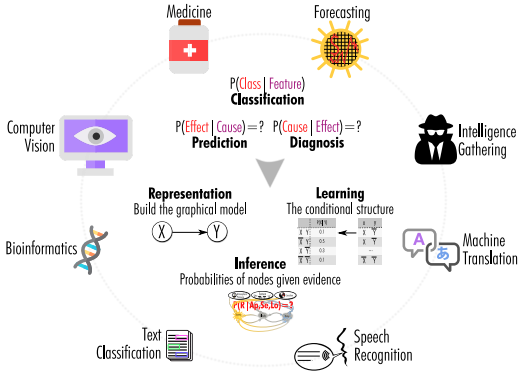

### A shared reasoning pattern

Several variables interact; only some are observed.

Use the observations to ask about a label, a future event or a hidden state.

Represent the joint, obtain its parameters, then compute a conditional probability.

## The same formalism serves different kinds of questions



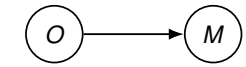

### A shared reasoning pattern

Several variables interact; only some are observed.

Use the observations to ask about a label, a future event or a hidden state.

Represent the joint, obtain its parameters, then compute a conditional probability.

## Does detected motion imply an occupied room?




At one instant, $O = 1$ means occupied and $M = 1$ means motion detected. Zero means the opposite state.

**The detector can make errors**

It can miss a person who remains still, or trigger in an empty room. Its reading is evidence, not the occupancy label.

$$\underbrace{P(M = 1 \mid O = 1)}_{\text{detector response}} \quad \text{versus} \quad \underbrace{P(O = 1 \mid M = 1)}_{\text{occupancy after a reading}}.$$

## Use the prior as well as the detector response

Suppose a room is occupied 20% of the time. The detector triggers in 90% of occupied cases and 10% of empty cases.

| Quantity | Probability | Value |
|---|---|---|
| Occupancy before the reading | $P(O = 1)$ | .20 |
| Detection when occupied | $P(M = 1 \mid O = 1)$ | .90 |
| Detection when empty | $P(M = 1 \mid O = 0)$ | .10 |

### After a positive reading: Bayes' rule

$$P(O = 1 \mid M = 1) = \frac{.90 \times .20}{.90 \times .20 + .10 \times .80} = \frac{18}{26} \approx .692.$$

## A Bayesian network specifies variables, parents and CPDs

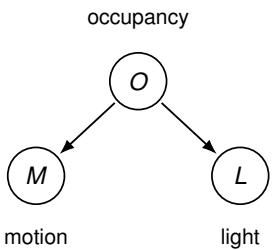
**Definition**

A Bayesian network is a directed acyclic graph with one random variable per node and one CPD per node, conditioned on all its parents.

## A Bayesian network specifies variables, parents and CPDs

**Definition**

A Bayesian network is a directed acyclic graph with one random variable per node and one CPD per node, conditioned on all its parents.

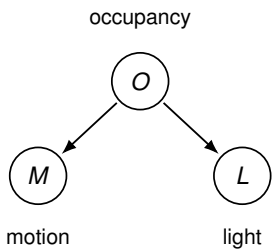



A root has no parents: its local distribution is unconditional.

## A Bayesian network specifies variables, parents and CPDs

**Definition**

A Bayesian network is a directed acyclic graph with one random variable per node and one CPD per node, conditioned on all its parents.




A root has no parents: its local distribution is unconditional.

$$\begin{aligned} \text{parents}(O) &= \varnothing, & P(O),\\ \text{parents}(M) &= \{O\}, & P(M \mid O),\\ \text{parents}(L) &= \{O\}, & P(L \mid O). \end{aligned}$$

## Independence removes unnecessary conditioning variables

The chain rule, for the order $O, M, L$, gives

$$P(O, M, L) = P(O)P(M \mid O)P(L \mid O, M).$$

**Conditional independence: $L \perp\!\!\!\perp M \mid O$**

Once occupancy is known, motion adds nothing about light.

## Independence removes unnecessary conditioning variables

The chain rule, for the order $O, M, L$, gives

$$P(O, M, L) = P(O)P(M \mid O)P(L \mid O, M).$$

**Conditional independence: $L \perp\!\!\!\perp M \mid O$**

Once occupancy is known, motion adds nothing about light.

$$P(L \mid O, M) = P(L \mid O).$$

## Independence removes unnecessary conditioning variables

The chain rule, for the order $O, M, L$, gives

$$P(O, M, L) = P(O)P(M \mid O)P(L \mid O, M).$$

**Conditional independence: $L \perp\!\!\!\perp M \mid O$**

Once occupancy is known, motion adds nothing about light.

$$P(L \mid O, M) = P(L \mid O).$$

Thus $P(O, M, L) = P(O)P(M \mid O)P(L \mid O)$: the unnecessary conditioning variable has disappeared.

## Read the same structure in either direction

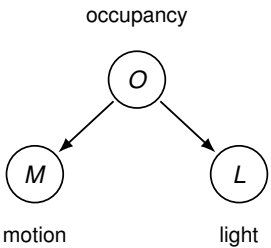
**Labelled DAG $\longleftrightarrow$ specified parent-wise factorisation**

One conditional factor per node; its conditioning variables are exactly its parents.

## Read the same structure in either direction

**Labelled DAG $\longleftrightarrow$ specified parent-wise factorisation**

One conditional factor per node; its conditioning variables are exactly its parents.



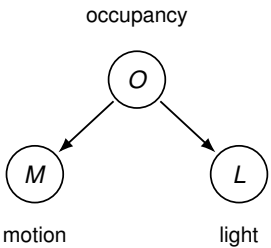

$\Longleftrightarrow$

$$P(O, M, L) = P(O)P(M \mid O)P(L \mid O).$$

## Read the same structure in either direction

**Labelled DAG $\longleftrightarrow$ specified parent-wise factorisation**

One conditional factor per node; its conditioning variables are exactly its parents.



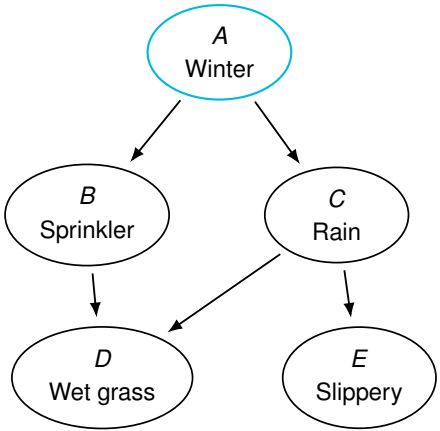

$\Longleftrightarrow$

$$P(O, M, L) = P(O)P(M \mid O)P(L \mid O).$$

For instance, $P(L \mid O)$ gives the edge $O \to L$, but no edge $M \to L$.

The graph specifies the product's form; the CPDs specify its values. The numerical joint need not have a unique graph.

## Apply the rule to the sprinkler network

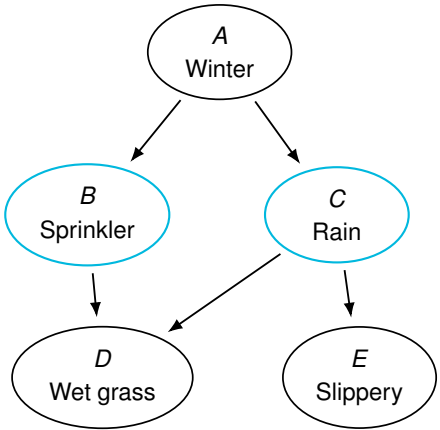



Each variable is binary: 1 means true, 0 means false.

**Build one factor for each node**

Root: $P(A)$.

## Apply the rule to the sprinkler network



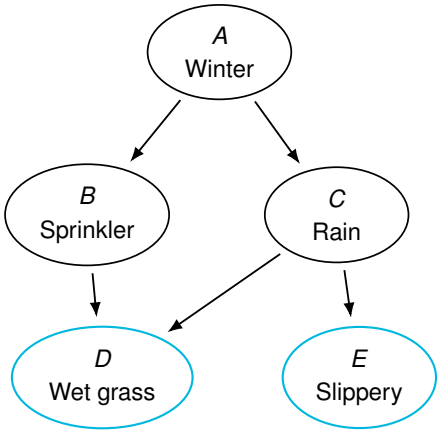

Each variable is binary: 1 means true, 0 means false.

**Build one factor for each node**

Root: $P(A)$.

Middle: $P(B \mid A)$ and $P(C \mid A)$.

## Apply the rule to the sprinkler network




Each variable is binary: 1 means true, 0 means false.

**Build one factor for each node**

Root: $P(A)$.

Middle: $P(B \mid A)$ and $P(C \mid A)$.

Leaves: $P(D \mid B, C)$ and $P(E \mid C)$.

$$P(A, B, C, D, E) = P(A)P(B \mid A)P(C \mid A) \times P(D \mid B, C)P(E \mid C).$$

## Each parent configuration defines one probability column

**A discrete CPD**

Fix the parents' states. The probabilities over the child's possible states sum to one.

## Each parent configuration defines one probability column

### A discrete CPD

Fix the parents' states. The probabilities over the child's possible states sum to one.

For wet grass $D$, the parents are sprinkler $B$ and rain $C$.

| $(B, C)$ | $(1,1)$ | $(1,0)$ | $(0,1)$ | $(0,0)$ |
|---|---|---|---|---|
| $P(D=1 \mid B, C)$ | .95 | .90 | .80 | .00 |
| $P(D=0 \mid B, C)$ | .05 | .10 | .20 | 1.00 |

## Each parent configuration defines one probability column

### A discrete CPD

Fix the parents' states. The probabilities over the child's possible states sum to one.

For wet grass $D$, the parents are sprinkler $B$ and rain $C$.

| $(B, C)$ | $(1,1)$ | $(1,0)$ | $(0,1)$ | $(0,0)$ |
|---|---|---|---|---|
| $P(D=1 \mid B, C)$ | .95 | .90 | **.80** | .00 |
| $P(D=0 \mid B, C)$ | .05 | .10 | .20 | 1.00 |

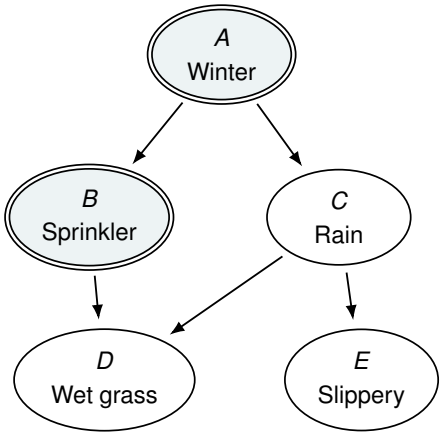
Example: with sprinkler off and rain present, wet grass has probability .80; the complementary probability is .20.

## The query changes; the underlying model does not

**Query, evidence and hidden variables**

The query is what we ask about; evidence gives known states. The remaining variables are unobserved.

## The query changes; the underlying model does not

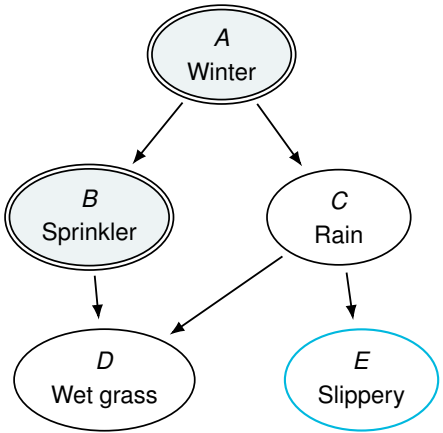



**Query, evidence and hidden variables**

The query is what we ask about; evidence gives known states. The remaining variables are unobserved.

Evidence: winter $A = 1$, sprinkler $B = 1$.

## The query changes; the underlying model does not




**Query, evidence and hidden variables**

The query is what we ask about; evidence gives known states. The remaining variables are unobserved.

Evidence: winter $A = 1$, sprinkler $B = 1$.

Query: road $E$; unobserved: rain $C$, wet grass $D$.

$$P(E = 1 \mid A = 1, B = 1).$$

Double outline: evidence. Coloured outline: query.

# Conditional independence

## Some observations become redundant after conditioning

### Conditional independence

$X \perp\!\!\!\perp Y \mid Z$: once $Z$ is known, observing $Y$ does not change the distribution of $X$.

$$P(X \mid Y, Z) = P(X \mid Z).$$

For the sprinkler model, $B \perp\!\!\!\perp E \mid A$: once winter is known, the sprinkler tells us nothing further about the road.

How can we recognise such statements directly in a graph?

## Observing a non-collider blocks its path

**Non-collider rule**

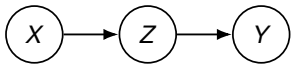
A path through a chain or a fork is blocked when the middle variable is observed.

## Observing a non-collider blocks its path

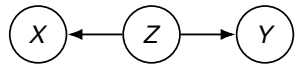
### Non-collider rule

A path through a chain or a fork is blocked when the middle variable is observed.

Chain




Fork




Middle variable unobserved: each path is active.

## Observing a non-collider blocks its path

**Non-collider rule**

A path through a chain or a fork is blocked when the middle variable is observed.




Middle variable observed: each path is blocked.

In either isolated three-node graph, $X \perp\!\!\!\perp Y \mid Z$.

Example: in $A \to C \to E$, knowing rain $C$ makes winter $A$ redundant for the road $E$. Check other paths in a larger graph.

## An observed effect can link its explanations

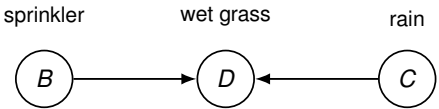
**Collider rule**

$X \rightarrow Z \leftarrow Y$ is blocked unless $Z$, or a descendant of $Z$, is observed.

## An observed effect can link its explanations

**Collider rule**

$X \rightarrow Z \leftarrow Y$ is blocked unless $Z$, or a descendant of $Z$, is observed.



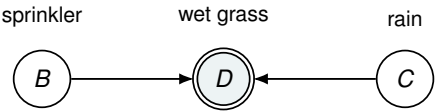

Wet grass unobserved: collider path blocked.

In the sprinkler network, keep winter observed to block the other path.

## An observed effect can link its explanations

**Collider rule**

$X \rightarrow Z \leftarrow Y$ is blocked unless $Z$, or a descendant of $Z$, is observed.




Wet grass observed: collider path active.

In the sprinkler network, keep winter observed to block the other path.

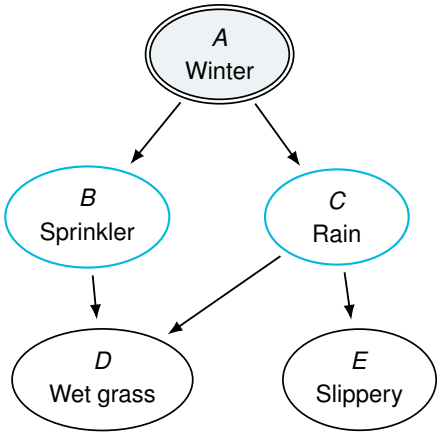
With wet grass observed, an active sprinkler can partly explain it, making rain less likely. We will calculate this **explaining-away** effect.

## D-separation checks all paths between two variables

**Formal criterion**

All paths between two variables must be blocked by the evidence for independence to be graphically guaranteed.

## D-separation checks all paths between two variables

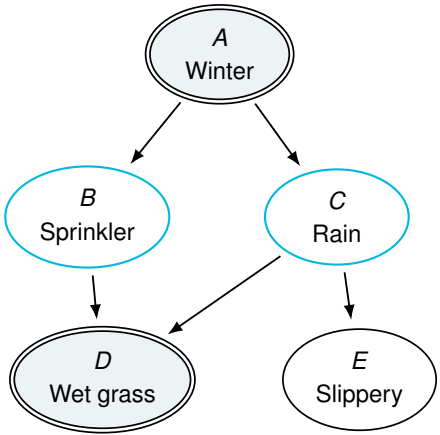



**Formal criterion**

All paths between two variables must be blocked by the evidence for independence to be graphically guaranteed.

Given $A$: the fork $B \leftarrow A \rightarrow C$ is blocked; the collider at $D$ is also blocked. Thus $B \perp\!\!\!\perp C \mid A$.

## D-separation checks all paths between two variables




**Formal criterion**

All paths between two variables must be blocked by the evidence for independence to be graphically guaranteed.

Given $A$: the fork $B \leftarrow A \rightarrow C$ is blocked; the collider at $D$ is also blocked. Thus $B \perp\!\!\!\perp C \mid A$.

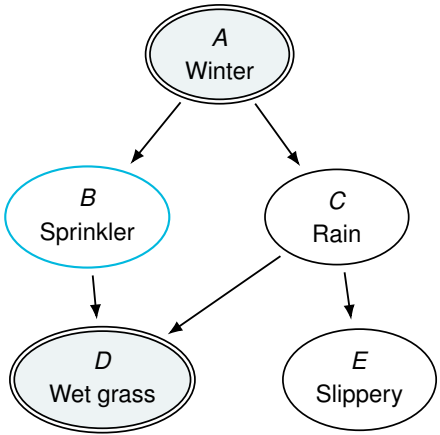
Also observe $D$: the path $B \rightarrow D \leftarrow C$ opens. Independence is no longer guaranteed.

## The Markov blanket contains parents, children and co-parents

**Condition on the blanket**

A node is independent of all remaining variables given its parents, children and children's other parents.

## The Markov blanket contains parents, children and co-parents

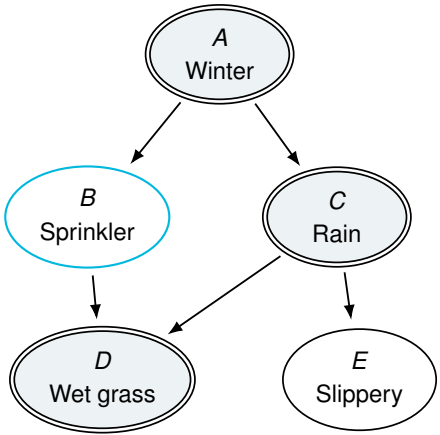



**Condition on the blanket**

A node is independent of all remaining variables given its parents, children and children's other parents.

For $B$: include parent $A$ and child $D$.

## The Markov blanket contains parents, children and co-parents




**Condition on the blanket**

A node is independent of all remaining variables given its parents, children and children's other parents.

For $B$: include parent $A$ and child $D$.

Include $C$, the other parent of $D$:

$$\mathrm{MB}(B) = \{A, C, D\}.$$

$$B \perp\!\!\!\perp E \mid A, C, D.$$

Rain belongs to the blanket even without an edge between rain and sprinkler.

## The graph guarantees some, not necessarily all, independences

### An independence map (I-map)

Let $I(G)$ be the independences implied by the graph, and $I(P)$ those true in the joint distribution.

$$I(G) \subseteq I(P).$$

D-separated variables are independent in every model with that factorisation.

An active path means independence is not guaranteed: particular CPDs may still make it hold.

## Different graphs can express the same independence structure

### Same labelled skeleton and no collider

$$X \longrightarrow Y \longrightarrow Z, \qquad X \longleftarrow Y \longleftarrow Z, \qquad X \longleftarrow Y \longrightarrow Z.$$

All three imply $X \perp\!\!\!\perp Z \mid Y$: they are **Markov equivalent**.

The collider $X \longrightarrow Y \longleftarrow Z$ is different: it implies $X \perp\!\!\!\perp Z$ before $Y$ is observed.

The correspondence is graph $\leftrightarrow$ chosen factorisation, not a unique graph for a joint. Arrows alone do not establish intervention effects.

# Inference: from a question to factor operations

## Independence tells us what matters; inference tells us how much

We know how to read the model. Now we want an actual probability, not only an independence statement.

### The road question

Given winter and an active sprinkler, compute both possible road states:

$$P(E \mid A = 1, B = 1).$$

Rain $C$ and wet grass $D$ are unobserved: consider their possible states rather than setting them to zero.

**Exact inference** computes this distribution for the specified model, up to numerical precision.

## A direct calculation sums over the hidden configurations

For query $Q$, evidence $\mathbf{e}$ and hidden variables $\mathbf{Z}$, form a factor

$$f(Q) = \sum_{\mathbf{Z}} P(Q, \mathbf{Z}, \mathbf{e}), \qquad P(Q \mid \mathbf{e}) = \frac{f(Q)}{\sum_q f(q)}.$$

**Road query: $a = b = 1$**

## A direct calculation sums over the hidden configurations

For query $Q$, evidence $\mathbf{e}$ and hidden variables $\mathbf{Z}$, form a factor

$$f(Q) = \sum_{\mathbf{Z}} P(Q, \mathbf{Z}, \mathbf{e}), \qquad P(Q \mid \mathbf{e}) = \frac{f(Q)}{\sum_q f(q)}.$$

### Road query: $a = b = 1$

$$f(e) = \sum_{c \in \{0,1\}} \sum_{d \in \{0,1\}} P(a, b, c, d, e).$$

## A direct calculation sums over the hidden configurations

For query $Q$, evidence $\mathbf{e}$ and hidden variables $\mathbf{Z}$, form a factor

$$f(Q) = \sum_{\mathbf{Z}} P(Q, \mathbf{Z}, \mathbf{e}), \qquad P(Q \mid \mathbf{e}) = \frac{f(Q)}{\sum_q f(q)}.$$

### Road query: $a = b = 1$

$$f(e) = \sum_{c \in \{0,1\}} \sum_{d \in \{0,1\}} P(a, b, c, d, e).$$

For each $e$, add the four $(c, d)$ cases: $(0,0), (0,1), (1,0), (1,1)$.

Enumeration is exact, but the joint is already a product of small factors. Variable elimination reuses that structure instead of rebuilding complete assignments.

## A factor has a scope, a domain and numerical values

### Definition

For a set of variables $S$, a factor is a nonnegative function

$$f : \underbrace{\prod_{X \in S} \mathcal{X}_X}_{\text{assignment domain}} \longrightarrow \mathbb{R}_{\geqslant 0}, \qquad \text{scope}(f) = S.$$

## A factor has a scope, a domain and numerical values

**Definition**

For a set of variables $S$, a factor is a nonnegative function

$$f : \underbrace{\prod_{X \in S} \mathcal{X}_X}_{\text{assignment domain}} \longrightarrow \mathbb{R}_{\geqslant 0}, \qquad \text{scope}(f) = S.$$

Think of a table: the variables index its entries; an assignment selects one number. Factors need not sum to one.

## A factor has a scope, a domain and numerical values

**Definition**

For a set of variables $S$, a factor is a nonnegative function

$$f : \underbrace{\prod_{X \in S} \mathcal{X}_X}_{\text{assignment domain}} \longrightarrow \mathbb{R}_{\geqslant 0}, \qquad \text{scope}(f) = S.$$

Think of a table: the variables index its entries; an assignment selects one number. Factors need not sum to one.

Example: $f(c, e) = P(E = e \mid C = c)$.

| $f(c,e)$ | $e=0$ | $e=1$ |
|---|---|---|
| $c=0$ | 1.00 | .00 |
| $c=1$ | .30 | .70 |

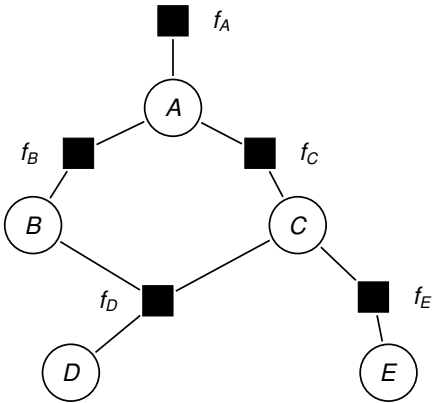
Scope $\{C, E\}$; domain $\{0,1\} \times \{0,1\}$.

## A factor graph makes the product explicit

**A bipartite representation**

Circles denote variables; squares denote factors. A link means that a variable belongs to a factor's scope.

## A factor graph makes the product explicit

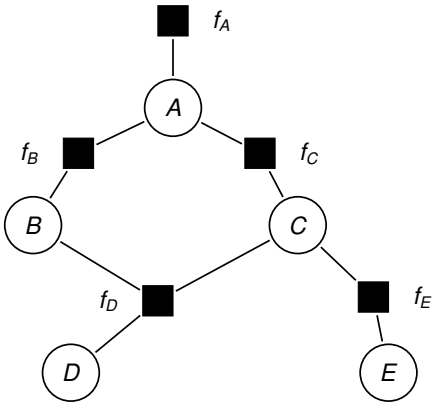



### A bipartite representation

Circles denote variables; squares denote factors. A link means that a variable belongs to a factor's scope.

The same product can be written $f_A f_B f_C f_D f_E$.

## A factor graph makes the product explicit




### A bipartite representation

Circles denote variables; squares denote factors. A link means that a variable belongs to a factor's scope.

The same product can be written $f_A f_B f_C f_D f_E$.

$$
\begin{aligned}
f_A(A) &= P(A), \\
f_B(A, B) &= P(B \mid A), \\
f_C(A, C) &= P(C \mid A), \\
f_D(B, C, D) &= P(D \mid B, C), \\
f_E(C, E) &= P(E \mid C).
\end{aligned}
$$

## Restriction fixes an argument and returns a new factor

**Restriction to $Z = z_0$**

$f'(\mathbf{y}) = f(\mathbf{y}, z_0)$; the scope changes from $S$ to $S \setminus \{Z\}$.

## Restriction fixes an argument and returns a new factor

**Restriction to $Z = z_0$**

$f'(\mathbf{y}) = f(\mathbf{y}, z_0)$; the scope changes from $S$ to $S \setminus \{Z\}$.

For this example, observe rain $C = 1$ in $f(C, E) = P(E \mid C)$.

| $f(c, e)$ | $e = 0$ | $e = 1$ |
|---|---|---|
| $c = 0$ | 1.00 | .00 |
| $c = 1$ | .30 | .70 |

## Restriction fixes an argument and returns a new factor

**Restriction to $Z = z_0$**

$f'(\mathbf{y}) = f(\mathbf{y}, z_0)$; the scope changes from $S$ to $S \setminus \{Z\}$.

For this example, observe rain $C = 1$ in $f(C, E) = P(E \mid C)$.

| $f(c,e)$ | $e=0$ | $e=1$ |
|---|---|---|
| $c=0$ | 1.00 | .00 |
| $c=1$ | .30 | .70 |

$\xrightarrow{C=1}$

| $e$ | $f'(e) = f(1,e)$ |
|---|---|
| 0 | .30 |
| 1 | .70 |

$$f : \mathcal{X}_C \times \mathcal{X}_E \to \mathbb{R}_{\geqslant 0} \quad \longrightarrow \quad f' : \mathcal{X}_E \to \mathbb{R}_{\geqslant 0}.$$

The selected value is stored in the entries; $C$ is no longer a free argument.

## Multiplication joins compatible assignments

### Factor product

$f'(\mathbf{x}) = (f_1 f_2)(\mathbf{x})$ has scope $\mathrm{scope}(f_1) \cup \mathrm{scope}(f_2)$. Shared variables must have the same state.

## Multiplication joins compatible assignments

**Factor product**

$f'(\mathbf{x}) = (f_1 f_2)(\mathbf{x})$ has scope $\mathrm{scope}(f_1) \cup \mathrm{scope}(f_2)$. Shared variables must have the same state.

For winter observed and rain unknown, use $f_1(c) = P(c \mid A = 1)$ and $f_2(c, e) = P(e \mid c)$.

## Multiplication joins compatible assignments

### Factor product

$f'(\mathbf{x}) = (f_1 f_2)(\mathbf{x})$ has scope $\mathrm{scope}(f_1) \cup \mathrm{scope}(f_2)$. Shared variables must have the same state.

For winter observed and rain unknown, use $f_1(c) = P(c \mid A = 1)$ and $f_2(c, e) = P(e \mid c)$.

| $c$ | $e$ | $f_1(c)$ | $f_2(c, e)$ | $f'(c, e) = f_1(c) f_2(c, e)$ |
|---|---|---|---|---|
| 0 | 0 | .20 | 1.00 | .20 |
| 0 | 1 | .20 | .00 | .00 |
| 1 | 0 | .80 | .30 | .24 |
| 1 | 1 | .80 | .70 | .56 |

Scope: $\{C, E\}$; domain: $\mathcal{X}_C \times \mathcal{X}_E$. The shared variable $C$ appears only once.

## Summing out a variable returns a factor over the others

### Marginalisation

$f''(\mathbf{y}) = \sum_{z \in \mathcal{X}_Z} f'(\mathbf{y}, z)$; its scope is $\mathrm{scope}(f') \setminus \{Z\}$.

## Summing out a variable returns a factor over the others

**Marginalisation**

$f''(\mathbf{y}) = \sum_{z \in \mathfrak{X}_Z} f'(\mathbf{y}, z)$; its scope is $\mathrm{scope}(f') \setminus \{Z\}$.

Keep the product $f'(C, E)$ from the preceding slide. Group the entries that agree on $E$.

| $c$ | $e$ | $f'(c, e)$ |
|---|---|---|
| 0 | 0 | .20 |
| 1 | 0 | .24 |
| 0 | 1 | .00 |
| 1 | 1 | .56 |

## Summing out a variable returns a factor over the others

**Marginalisation**

$f''(\mathbf{y}) = \sum_{z \in \mathcal{X}_Z} f'(\mathbf{y}, z)$; its scope is $\mathrm{scope}(f') \setminus \{Z\}$.

Keep the product $f'(C, E)$ from the preceding slide. Group the entries that agree on $E$.

| $c$ | $e$ | $f'(c,e)$ |
|---|---|---|
| 0 | 0 | .20 |
| 1 | 0 | .24 |
| 0 | 1 | .00 |
| 1 | 1 | .56 |

$\xrightarrow{\sum_C}$

| $e$ | $f''(e)$ |
|---|---|
| 0 | $.20 + .24 = .44$ |
| 1 | $.00 + .56 = .56$ |

Domain: $\mathcal{X}_C \times \mathcal{X}_E \longrightarrow \mathcal{X}_E$. Unlike restriction, *all* rain states contribute.

## Normalisation changes values, not the factor domain

### Divide by the positive total mass

$f_{\text{post}}(E) = \dfrac{f'''(E)}{\sum_e f'''(e)}$. The scope and domain remain $\{E\}$ and $\mathcal{X}_E$.

## Normalisation changes values, not the factor domain

**Divide by the positive total mass**

$f_{\text{post}}(E) = \dfrac{f'''(E)}{\sum_e f'''(e)}$. The scope and domain remain $\{E\}$ and $\mathcal{X}_E$.

For the road query, $P(a, b) = .6(.2) = .12$. Multiply the previous result by this evidence weight: $f'''(E) = .12 f''(E)$.

## Normalisation changes values, not the factor domain

**Divide by the positive total mass**

$f_{\text{post}}(E) = \dfrac{f'''(E)}{\sum_e f'''(e)}$. The scope and domain remain $\{E\}$ and $\mathcal{X}_E$.

For the road query, $P(a, b) = .6(.2) = .12$. Multiply the previous result by this evidence weight: $f'''(E) = .12 f''(E)$.

| $e$ | $f''(e)$ | $f'''(e)$ | $f_{\text{post}}(e)$ |
|---|---|---|---|
| 0 | .44 | .0528 | $.0528/.12 = .44$ |
| 1 | .56 | .0672 | $.0672/.12 = .56$ |

The normaliser .12 is the evidence probability; the result is $P(E \mid a, b)$.

## Variable elimination: collect, multiply, sum and replace

First restrict every CPD to the observed evidence. Eliminate only variables that are neither queried nor observed.

### One elimination step for a variable $Z$

Collect all current factors $f_1, \ldots, f_m$ containing $Z$. Multiply them, sum out $Z$, and replace them by $f'$.

$$f'(\mathbf{y}) = \sum_{z} (f_1 \cdots f_m)(\mathbf{y}, z).$$

## Variable elimination: collect, multiply, sum and replace

First restrict every CPD to the observed evidence. Eliminate only variables that are neither queried nor observed.

### One elimination step for a variable $Z$

Collect all current factors $f_1, \ldots, f_m$ containing $Z$. Multiply them, sum out $Z$, and replace them by $f'$.

$$f'(\mathbf{y}) = \sum_z (f_1 \cdots f_m)(\mathbf{y}, z).$$

The output scope is the union of the collected scopes, minus $Z$. Leave the other factors unchanged.

## Variable elimination: collect, multiply, sum and replace

First restrict every CPD to the observed evidence. Eliminate only variables that are neither queried nor observed.

### One elimination step for a variable $Z$

Collect all current factors $f_1, \dots, f_m$ containing $Z$. Multiply them, sum out $Z$, and replace them by $f'$.

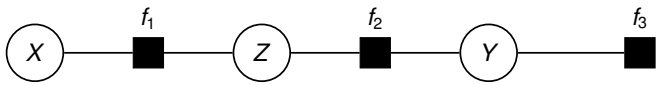
$$f'(\mathbf{y}) = \sum_z (f_1 \cdots f_m)(\mathbf{y}, z).$$

The output scope is the union of the collected scopes, minus $Z$. Leave the other factors unchanged.

Repeat in the chosen order. Finally multiply the remaining factors and normalise over the query states.

## Illustration: replace only the factors containing the variable

Eliminate $Z$ from $f_1(X, Z)$, $f_2(Z, Y)$ and $f_3(Y)$.

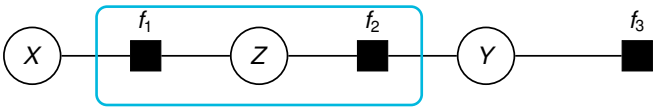



Initial factor collection; $Z$ is to be eliminated.

### Keep the arguments still needed

## Illustration: replace only the factors containing the variable

Eliminate $Z$ from $f_1(X, Z)$, $f_2(Z, Y)$ and $f_3(Y)$.



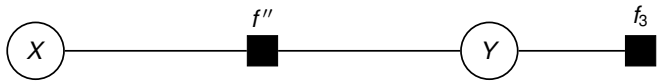

Collect $f_1$, $f_2$: their product has scope $\{X, Z, Y\}$.

**Keep the arguments still needed**

$f'(X, Z, Y) = f_1(X, Z) f_2(Z, Y)$.

## Illustration: replace only the factors containing the variable

Eliminate $Z$ from $f_1(X, Z)$, $f_2(Z, Y)$ and $f_3(Y)$.




The new factor connects $X$, $Y$; $f_3$ is unchanged.

### Keep the arguments still needed

$f'(X, Z, Y) = f_1(X, Z) f_2(Z, Y)$.

$f''(X, Y) = \sum_Z f'(X, Z, Y)$. Replace $f_1, f_2$ by $f''$; retain $f_3(Y)$.

$f''$ must stay inside a later sum over $X$ or $Y$.

## Worked elimination: infer rain from winter and wet grass

Query $C$; observe $A = 1, D = 1$; eliminate $E$ then $B$.

### Restrict the five original CPD factors

$f_A = .60$, $f_B(B) = P(B \mid A = 1)$, $f_C(C) = P(C \mid A = 1)$.

$f_D(B, C) = P(D = 1 \mid B, C)$, $f_E(C, E) = P(E \mid C)$.

## Worked elimination: infer rain from winter and wet grass

Query $C$; observe $A = 1, D = 1$; eliminate $E$ then $B$.

### Restrict the five original CPD factors

$f_A = .60$, $f_B(B) = P(B \mid A = 1)$, $f_C(C) = P(C \mid A = 1)$.

$f_D(B, C) = P(D = 1 \mid B, C)$, $f_E(C, E) = P(E \mid C)$.

| $f_D(b, c)$ | $c = 0$ | $c = 1$ |
|---|---|---|
| $b = 0$ | .00 | .80 |
| $b = 1$ | .90 | .95 |

$f_B = (.80, .20), \quad f_C = (.20, .80).$

## Worked elimination: infer rain from winter and wet grass

Query $C$; observe $A = 1, D = 1$; eliminate $E$ then $B$.

### Restrict the five original CPD factors

$f_A = .60$, $f_B(B) = P(B \mid A = 1)$, $f_C(C) = P(C \mid A = 1)$.

$f_D(B, C) = P(D = 1 \mid B, C)$, $f_E(C, E) = P(E \mid C)$.

| $f_D(b, c)$ | $c = 0$ | $c = 1$ |
|---|---|---|
| $b = 0$ | .00 | .80 |
| $b = 1$ | .90 | .95 |

$f_B = (.80, .20), \quad f_C = (.20, .80).$

Only $f_E$ contains $E$, so $f'(C) = \sum_E f_E(C, E) = 1$ for each $C$. The remaining collection is $\{f_A, f_B, f_C, f_D, f'\}$.

## Eliminate the sprinkler, then normalise the rain factor

Collect $f_B(B)$ and $f_D(B, C)$. Replace them by the factor

$$f''(C) = \sum_B f_B(B) f_D(B, C).$$

## Eliminate the sprinkler, then normalise the rain factor

Collect $f_B(B)$ and $f_D(B, C)$. Replace them by the factor

$$f''(C) = \sum_B f_B(B) f_D(B, C).$$

| $c$ | $f''(c)$ |
|---|---|
| 0 | $.80(.00) + .20(.90) = .18$ |
| 1 | $.80(.80) + .20(.95) = .83$ |

## Eliminate the sprinkler, then normalise the rain factor

Collect $f_B(B)$ and $f_D(B, C)$. Replace them by the factor

$$f''(C) = \sum_B f_B(B) f_D(B, C).$$

| $c$ | $f''(c)$ |
|---|---|
| 0 | $.80(.00) + .20(.90) = .18$ |
| 1 | $.80(.80) + .20(.95) = .83$ |

**Multiply the remaining factors; then normalise**

$f'''(C) = f_A f_C(C) f'(C) f''(C) = (.0216, .3984)$.

$P(C = 1 \mid A = 1, D = 1) = .3984/(.0216 + .3984) \approx .949$.

## Add sprinkler evidence without changing the model

We just found $P(C = 1 \mid A = 1, D = 1) \approx .949$. Now also observe $B = 1$.

**Predict before calculating**

The sprinkler can explain some of the wet grass. Should the rain posterior rise or fall?

## Add sprinkler evidence without changing the model

We just found $P(C = 1 \mid A = 1, D = 1) \approx .949$. Now also observe $B = 1$.

**Predict before calculating**

The sprinkler can explain some of the wet grass. Should the rain posterior rise or fall?

$$P(C = 1 \mid A = 1, D = 1, B = 1) = \frac{.80(.95)}{.80(.95) + .20(.90)} = \frac{38}{47} \approx .809.$$

## Add sprinkler evidence without changing the model

We just found $P(C = 1 \mid A = 1, D = 1) \approx .949$. Now also observe $B = 1$.

**Predict before calculating**

The sprinkler can explain some of the wet grass. Should the rain posterior rise or fall?

$$P(C = 1 \mid A = 1, D = 1, B = 1) = \frac{.80(.95)}{.80(.95) + .20(.90)} = \frac{38}{47} \approx .809.$$

Observing the sprinkler replaces its summation by restriction. The rain posterior falls in this model; the graph and CPDs have not changed.

## Exactness and computational cost are different questions

| Method | Work performed | Result |
|---|---|---|
| Enumeration | Sum complete assignments | Exact |
| Variable elimination | Reuse intermediate factors | Exact |
| Sampling | Estimate from simulated cases | Approximate |

**For variable elimination, intermediate scopes determine the cost**

The elimination order can change table sizes without changing the answer. Finite-sample estimates introduce sampling error instead.

So far, every calculation assumed that the graph and CPD values were already known.

# Learning and checking a model

## Learning estimates the tables; inference uses the tables

**Until now, the graph and its probabilities were supplied**

For the occupancy application we keep the graph fixed, but estimate its CPD entries from labelled observations.

| Task | What is unknown? |
|---|---|
| Parameter learning | The entries $\theta$ of the CPDs, estimated from training records. |
| Inference | A variable's state in a new case, given that case's evidence and the fitted CPDs. |

First define the variables and their states; then estimate, predict and evaluate.

## Use occupancy as the parent of the sensor features

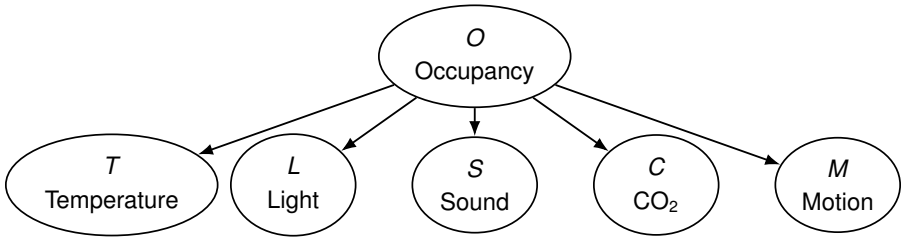
### Naive-Bayes factorisation

Each sensor feature is independent of the others, given occupancy $O$.

## Use occupancy as the parent of the sensor features

### Naive-Bayes factorisation

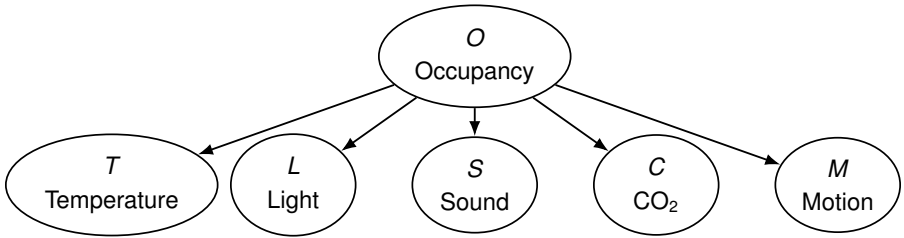
Each sensor feature is independent of the others, given occupancy $O$.



## Use occupancy as the parent of the sensor features

### Naive-Bayes factorisation

Each sensor feature is independent of the others, given occupancy $O$.




$$P(O, T, L, S, C, M) = P(O)P(T \mid O)P(L \mid O)P(S \mid O)P(C \mid O)P(M \mid O).$$

## A discrete CPD needs a finite set of states

Our implementation uses tables, so continuous sensor measurements must first be mapped to discrete states.

| Quantity | Recorded form | Model states |
|---|---|---|
| Occupancy | Number of people | Empty / occupied |
| Motion | Detector readings | No motion / motion |
| Temperature, light, sound, $CO_2$ | Numerical measurements | Low / medium / high |

**Discretisation: the same rule must apply to later readings**

For cut points $a < b$: low if $x < a$, medium if $a \leqslant x < b$, high if $x \geqslant b$. Learn $a, b$ from training data only.

More states retain detail but require more probability entries and observations.

## Separate fitting from evaluation before estimating anything

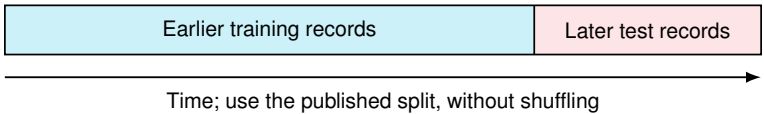
**Training data build the model; test data check it**

Define bins and estimate CPDs using the training records. Freeze those choices before evaluating later predictions.

## Separate fitting from evaluation before estimating anything

**Training data build the model; test data check it**

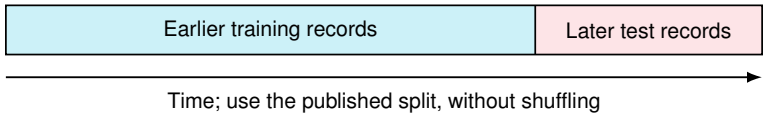
Define bins and estimate CPDs using the training records. Freeze those choices before evaluating later predictions.



## Separate fitting from evaluation before estimating anything

**Training data build the model; test data check it**

Define bins and estimate CPDs using the training records. Freeze those choices before evaluating later predictions.




Predict occupancy from test sensor readings, then compare with test labels.

Using test records to choose bins or probabilities is **data leakage**: evaluation information has entered fitting.

## Maximum likelihood chooses probabilities that explain the records

Suppose ten occupied training records contain eight motion detections and two non-detections. Let $\theta = P(M = 1 \mid O = 1)$.

**Likelihood: score a candidate parameter using the observations**

$$L(\theta) = \theta^8(1-\theta)^2, \qquad \widehat{\theta} = \arg\max_{\theta \in [0,1]} L(\theta) = \frac{8}{10}.$$

The observations are fixed; $\theta$ varies. The maximizing estimate is the observed conditional frequency.

Apply the same counting principle separately to each parent configuration.

## Count within each parent configuration, then normalise

### Complete-data maximum likelihood

$\widehat{P}(X = k \mid \text{parents} = j) = N_{jk} / \sum_l N_{jl}$. Group by the parents before counting the child's states.

## Count within each parent configuration, then normalise

### Complete-data maximum likelihood

$\widehat{P}(X = k \mid \text{parents} = j) = N_{jk} / \sum_l N_{jl}$. Group by the parents before counting the child's states.

| Count | $O = 0$ | $O = 1$ |
|---|---|---|
| $M = 0$ | 18 | 2 |
| $M = 1$ | 2 | 8 |
| Total | 20 | 10 |

## Count within each parent configuration, then normalise

### Complete-data maximum likelihood

$\widehat{P}(X = k \mid \text{parents} = j) = N_{jk} / \sum_l N_{jl}$. Group by the parents before counting the child's states.

| Count | $O = 0$ | $O = 1$ |
|---|---|---|
| $M = 0$ | 18 | 2 |
| $M = 1$ | 2 | 8 |
| Total | 20 | 10 |

$\longrightarrow$

| $\widehat{P}(M \mid O)$ | $O = 0$ | $O = 1$ |
|---|---|---|
| $M = 0$ | .90 | .20 |
| $M = 1$ | .10 | .80 |

For $O = 1$, divide by 10, not 30. If a parent configuration never occurs, these data do not determine its CPD column.

## Smoothing gives support to states not observed in a small sample

### Pseudocount rule

Add positive prior counts to the observed state counts, then normalise within that parent configuration.

Imagine eight occupied records, all with detected motion.

| State | $M = 0$ | $M = 1$ |
|---|---|---|
| Observed counts | 0 | 8 |

## Smoothing gives support to states not observed in a small sample

### Pseudocount rule

Add positive prior counts to the observed state counts, then normalise within that parent configuration.

Imagine eight occupied records, all with detected motion.

| State | $M = 0$ | $M = 1$ |
|---|---|---|
| Observed counts | 0 | 8 |
| Add one to each count | 1 | 9 |

## Smoothing gives support to states not observed in a small sample

### Pseudocount rule

Add positive prior counts to the observed state counts, then normalise within that parent configuration.

Imagine eight occupied records, all with detected motion.

| State | $M = 0$ | $M = 1$ |
|---|---|---|
| Observed counts | 0 | 8 |
| Add one to each count | 1 | 9 |
| Normalise by the new total | .10 | .90 |

Zero observations need not imply physical impossibility. The next slide explains this rule as a prior over the CPD probabilities.

## A prior describes uncertainty about the CPD probabilities

For one CPD column, $\theta_k$ is the unknown probability of child state $k$.

### Prior support plus data support

$\alpha_k > 0$ is prior support for state $k$; $N_k$ is its observed count.

$$\widehat{\theta}_k = \frac{N_k + \alpha_k}{\sum_l (N_l + \alpha_l)}.$$

Larger pseudocounts give the prior more influence, especially with little data.

A **Dirichlet prior** is a distribution over the probability vector $(\theta_1, \ldots, \theta_K)$. The estimate above is its updated **posterior mean**, not its most probable vector (MAP).

## A baseline makes the value of the sensors visible

For each held-out record $i$, produce an occupancy probability $p_i$, then compare it with the recorded label $y_i \in \{0, 1\}$.

| Condition | Information available to the predictor |
|---|---|
| Prior only | Training occupancy frequency; no sensor reading. |
| Full model | All sensor readings for that test record. |
| Sensor outage | The same fitted model and record, with one sensor omitted. |

### A controlled comparison

Use the same test records and scoring rule. For the full-model versus outage comparison, keep the fitted parameters fixed and change only the available evidence.

## The Brier score measures error in probability predictions

Compare each test probability $p_i$ with the observed occupancy label $y_i \in \{0, 1\}$.

For $N$ test records,

$$\text{Brier} = \frac{1}{N} \sum_{i=1}^{N} (p_i - y_i)^2.$$

| Label $y$ | Prediction $p$ | $(p - y)^2$ |
|---|---|---|
| 0 | .90 | .81 |
| 0 | .10 | .01 |
| 1 | .80 | .04 |

**Lower is better on the same test records**

A confident wrong probability receives a large penalty. Average the squared error across the test records to compare the conditions.

## A threshold converts probabilities into decisions

Choose a threshold $\tau$: flag occupied when $p \geqslant \tau$.

| Decision | Actually empty | Actually occupied |
|---|---|---|
| Flag empty | Correct empty | False negative |
| Flag occupied | False positive | Correct occupied |

**A confusion matrix counts these four outcomes**

It distinguishes false occupied flags from missed occupied rooms. Changing $\tau$ can change those counts without changing any probability prediction.

Next we will choose $\tau$ from the relative costs of the two errors, rather than assuming 0.5.

## An unavailable sensor is marginalised, not set to zero

**Two distinct evidence sets**

No motion detected: include $M = 0$.
Motion sensor unavailable: leave $M$ unobserved.

If $\mathbf{e}$ denotes the other readings,

$$P(O, \mathbf{e}) = \sum_{m} P(O, \mathbf{e}, M = m).$$

Keep the fitted graph and CPDs unchanged. This tests the effect of unavailable evidence, not a newly trained model.

With all sensor readings unavailable, the model returns its fitted prior $P(O)$.

# From probabilities to engineering decisions

## State the consequences of the two kinds of error

Suppose a missed occupied room costs five units, while a false occupied flag costs one. Correct decisions cost zero.

| Decision | Actually empty | Actually occupied |
|---|---|---|
| Flag empty | 0 | 5 |
| Flag occupied | 1 | 0 |

**A decision model, in addition to the probability model**

These declared costs express which mistakes matter more. They need not favour the most probable label.

## Choose the action with the lower expected loss

Let $p = P(O = 1 \mid \text{evidence})$. Then

$$L(\text{flag occupied}) = 1 - p, \qquad L(\text{flag empty}) = 5p.$$

**Compare the two actions**

$$1 - p < 5p \iff p > \frac{1}{6}.$$

At $p = .30$, the losses are .70 and 1.50: flag occupied.

At equality either action minimises loss; our practical uses occupied as the tie decision, so its rule is $p \geqslant 1/6$.

## A temporal model carries information from the previous state

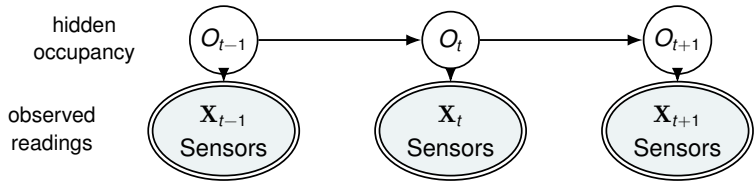
### A first-order hidden Markov model

The current hidden state depends on the previous state. Current observations depend on the current hidden state.

## A temporal model carries information from the previous state

### A first-order hidden Markov model

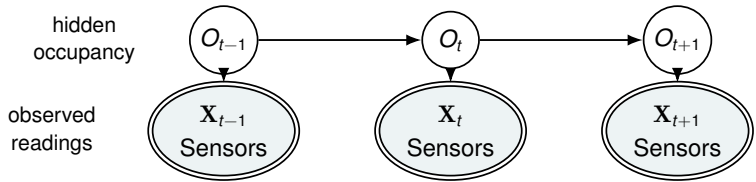
The current hidden state depends on the previous state. Current observations depend on the current hidden state.



## A temporal model carries information from the previous state

### A first-order hidden Markov model

The current hidden state depends on the previous state. Current observations depend on the current hidden state.




For room monitoring, $O_t$ is occupancy at time $t$ and $\mathbf{X}_t$ is its sensor vector. Empty and occupied are values of $O_t$, not separate variable nodes.

## Filtering and decoding answer different temporal questions

### Filtering: is the room occupied now?

$$P(O_t \mid \mathbf{X}_{1:t}).$$

Only readings available up to the current instant are used.

### Decoding: which state sequence best explains the recordings?

$$\arg\max_{o_{1:T}} P(o_{1:T} \mid \mathbf{X}_{1:T}).$$

For a standard HMM, Viterbi finds a most probable complete sequence exactly.

## Connect the model, its evidence and the engineering conclusion

### The complete reasoning chain

Choose variables and independence assumptions.
Read the graph as a factorisation and compute posteriors.
Estimate the CPDs from training data; evaluate on held-out records.
Use explicit error costs to interpret a decision.

Would the result survive a different room, dependent sensors or changed conditions?

A correct probability calculation is necessary, but not sufficient, for deployment.